不相交集合：只合併，不拆分

同一組元素分成哪些集合，會隨合併改變；**查詢要回答的是「兩個元素現在是否同組」**。

本章要解決的問題	對應方法
如何代表一個集合？	串列的集合紀錄，或森林的根
合併時要改多少資料？	短串列接長串列；低 rank 根接高 rank 根
查找為何愈做愈便宜？	路徑壓縮，加上整段操作的攤銷分析

先備知識：指標、根樹、漸近記號，以及位能法。rank、level、iter 都會在本章定義。

三個操作，維護同一份集合分割

集合分割（partition）要求：每個已建立元素恰屬一組；各組非空且兩兩不交。

操作	輸入前提	結果
MAKE-SET(x)	x 尚未建立	建立單點集合 {x}
FIND(x)	x 已存在	回傳 x 所在集合的代表
UNION(x,y)	x、y 已存在	合併兩者所在集合；同組時不變

代表是識別集合的代號，不保證是最小鍵。 同組元素的 FIND 必須回傳同一代表；不同組則回傳不同代表。

呼叫次數與成功合併，要分開計算

本章先建立 n 個單點。m 計入使用者呼叫的 MAKE-SET、FIND、UNION，所以 m≥n；UNION 內部呼叫的 FIND 不再重複加進 m。

名稱	規格與計數
成功合併	集合數減一，因此最多 n−1 次
UNION 呼叫	可以重複，次數不受 n−1 限制
LINK(r,s)	只接受兩個**不同的根**，不接受任意元素

> UNION 先 FIND 兩端；代表相同就返回。**不能再 LINK 同一根，也不能因此增加 rank。**

FIND 可以改 parent，但不改集合分割。刪邊、拆集合與撤銷合併，需要另外的結構與成本分析。

無向連通：新邊會合併哪些組？

先為每個頂點 MAKE-SET，再逐邊 UNION。圖解使用這一串輸入：ab、cd、bc、ab、ef、de。

邊串流：partition 的每次改變

每列為處理新邊後的partition；第4列重複邊不改集合數。

			集合數
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

成功5次合併：6→5→4→3→3→2→1；全部6次 UNION 仍各需做代表檢查。

第四條 ab 是重複邊：呼叫照算，集合不再改變。 六次 UNION 中只有五次成功合併。

為什麼這個方法恰好得到連通分量？

不變量：處理過的邊所形成的子圖，其連通分量恰好等於目前的集合分割。

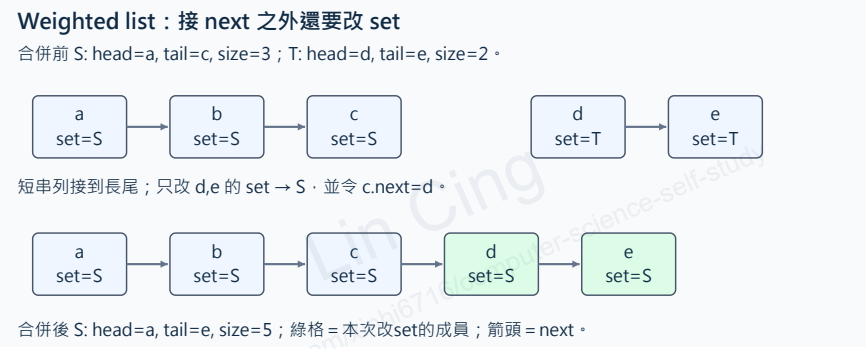
步驟	圖的連通性	集合分割
還沒有邊	每個頂點獨立	每點各成一組
新邊兩端已同組	原本就連通	UNION 不改分割
新邊兩端不同組	這條邊接起兩個分量	UNION 恰好合併這兩組

由歸納可知，最後 FIND(u)=FIND(v) 若且唯若存在無向 u–v 路徑。

靜態圖可用 DFS/BFS 在 Θ(n+|E|) 內找分量；DSU 也能處理**逐步加邊的連通查詢**。有向圖忽略方向後只得到弱連通分量，不能用來求 SCC。

串列表示：接上尾端，還沒完成合併

每組有一份集合紀錄（set record）：head、tail、size。每個成員另存 next 與 set 指標；FIND(x) 直接回傳 x.set.head，因此是 O(1)。



> a→b→c 與 d→e 合併後，tail 由 c 改為 e、size 由 3 改為 5。**d、e 的 set 也要改指新的紀錄**，FIND 才會回傳 a。

加權串列的 UNION，逐項做完

令 S=x.set、T=y.set。同組直接返回；不同組時，讓 S 是較長的串列，同長保留第一組。

次序	修改	目的
1	S.tail.next=T.head	把兩串列接起來
2	保存 T 的原 head；更新 S.tail、S.size	保留走訪起點，更新集合紀錄
3	沿 T 的 next，把每個成員的 set 改為 S	讓短串列的 FIND 改回傳 S.head
4	完成上述改寫後才廢棄 T 紀錄	不留下指向失效紀錄的成員

走訪時不改 T 內部的 next，因而不會遺失後繼。成本為 $\Theta(1+\min(|S|,|T|))$，**有 tail 只讓接線是 O(1)，不是整個 UNION。**

不加權，為什麼會退化成平方成本？

若固定把第二組接到第一組，依序做 UNION(x_2,x_1)、UNION(x_3,x_2)、…，每次新單點都要吸收舊長串列。

合併	必須重寫 set 的成員數
UNION(x_2,x_1)	1
UNION(x_3,x_2)	2
…	…
UNION(x_n,x_{n-1})	n−1

總改寫數為 $n(n-1)/2$。外部只有 n 次建立與 n−1 次合併，即 $m=2n-1$，實際成本卻是 $\Theta(n^2)$。

每次移短組，同一成員最多搬幾次？

固定一個成員 x。每次它的 set 被改寫，表示它在較短的集合；合併後所在組的大小**至少加倍**。

改 k 次後至少有 2^k 個成員，而總數不超過 n，所以 $k \le \lfloor \log_2 n \rfloor$。加總所有成員的改寫，再加上 m 次呼叫的常數開銷，得到 **$O(m+n \log n)$**；涵蓋小 n 可寫 $O(m+n \log(n+1))$。

要分清楚的量	結論
整段操作總成本	$O(m+n \log n)$
單次合併兩個各 n/2 的集合	仍需 $\Theta(n)$
$n=2^k$，同大小集合分輪合併	每輪改 n/2 筆，共 $(n/2)\log_2 n$ 筆

> 攤銷總界不能直接當成單次最壞界。短組策略也不保證所有序列都只有線性成本。

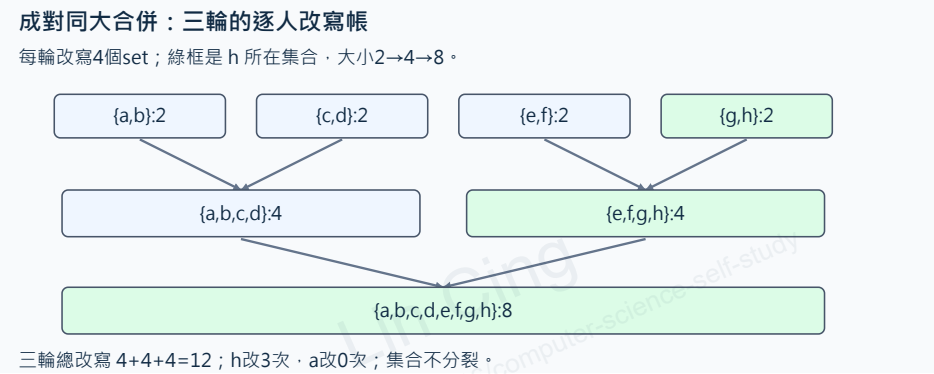
八元素：每次合併到底改了誰？

同長保留第一組。這一組操作稍後也會用來比較森林表示。

UNION	原大小	改 set 的成員	新代表／大小	累計改寫
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

h 的代表依 h→g→e→a，所屬組大小由 1→2→4→8；a 一次也不用搬。**合計 12 次，不代表每人都搬三次。**

用同一張圖，對照逐輪成本與個別成員



每輪的總改寫數與「固定一人搬幾次」是不同分母。表格逐輪加總為 12；h 個人則恰好改寫三次。

若反過來每次只把新單點接到大集合，總改寫可以只有 n−1 次；這不影響上界，也不代表所有序列皆線性。

森林表示：先找根，再接線

每個節點存 parent、rank。MAKE-SET(x) 設 parent[x]=x、rank[x]=0；根的 parent 指向自己。

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    若 r==s: 回傳 r
    回傳 LINK(r,s)
LINK(r,s):                    // r,s 是不同的根
    若 rank[r]<rank[s]: 交換 r,s
    parent[s]=r
    若 rank[r]==rank[s]: rank[r]+=1
    回傳 r
```

LINK 只接受兩個不同的根，所以不會形成環，也不會拆開舊集合。不同 rank 不加一；相同 rank 只讓保留根加一。

串列比較 size；森林比較 rank，後者不是精確的成員數。

兩趟 FIND：先找到根，再沿原路改接

從 x 出發，沿 parent 往上走，遇到自指的根就停。第一趟只找根 r，不改指標；第二趟再把**原查找路徑**上的每個節點直接接到 r。

```
FIND(x):
    r=x
    當 parent[r]!=r: r=parent[r]
    當 parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    回傳 r
```

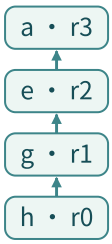
old 保存的是「覆寫前要走的下一站」。若改用 x=parent[x]，讀到的已是剛寫入的根，便跳過中間節點。遞迴版 parent[x]=FIND(parent[x]) 則在呼叫回傳時逐點改接。

兩趟迭代額外空間為 O(1)；遞迴堆疊與原路徑高度同階。走 s 點需 Θ(s)，只改本次路徑，不改 rank。

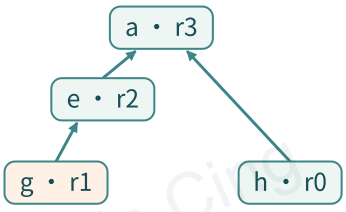
找根正確，不代表整條路徑都已壓縮

箭頭由**子指向父**；只畫 h→g→e→a，根 a 自指，其他節點不變。

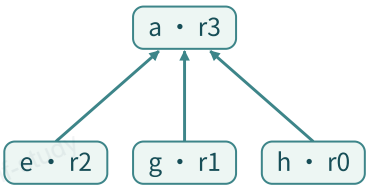
原路徑



覆寫後直接跳根



保存舊父，完成壓縮



第二趟目前節點	先存 old	再改 parent	繼續到
h	g	a	g
g	e	a	e
e	a	a（不變）	a，然後停止

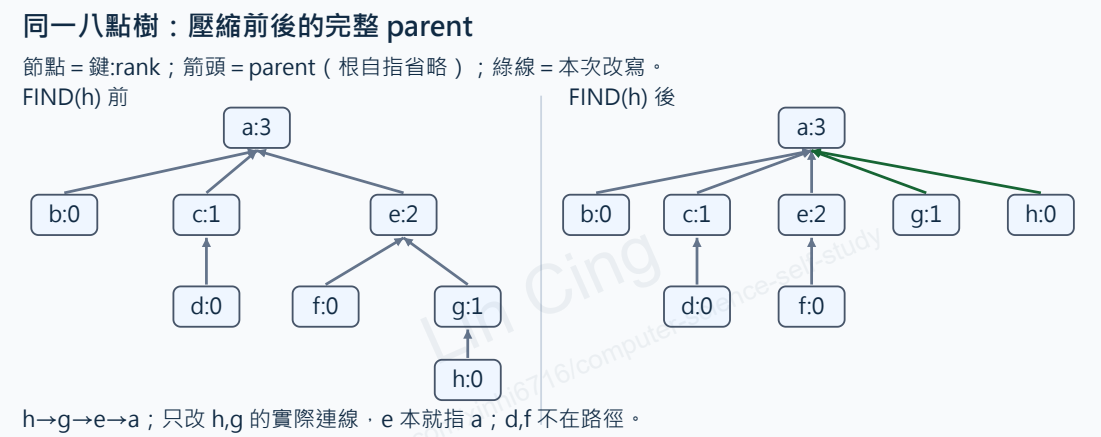
同一八元素的 rank 與壓縮前後

沿用前面七次 UNION；同 rank 時保留第一個根。先記錄每個節點的 rank 與 parent，再看 FIND(h) 改了哪些欄位。

點	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
FIND(h)前parent	a	a	a	c	a	e	e	g
FIND(h)後parent	a	a	a	c	a	e	a	a

- 第一趟走 h→g→e→a，共三條父邊。第二趟把沿途的 parent 設為 a。
- 真正改變的是 **h、g 的 parent**；e 原本就接 a，其他節點不在路徑上。
- **rank 全部不變**。它記錄合併歷史，不能在壓縮後改填目前高度。

看圖核對：改接路徑，保留其餘分支



> 再查 h 只走一條邊；查 d 仍走 d→c→a。**一次 FIND 只壓縮自己的查找路徑。**

Rank 為什麼能限制樹高？

rank 不是「目前高度」，但能當作它的**上界**。對操作次數歸納：

操作	樹高如何改變	為何仍有 height ≤ rank
建立單點	高度為 0	初始 rank 也是 0
不同 rank 合併	低 rank 的根接到高 rank 的根	低 rank 至少差 1，多接一條邊仍不超過大 rank
相同 rank 合併	高度最多增加 1	保留根的 rank 同時加 1
路徑壓縮	改接到祖先，只會縮短路徑	rank 不變，上界仍成立

> **壓縮後不重算 rank。** 例如根的 rank=2，壓縮後高度可以是 1；兩者不必相等。

Rank 也記得合併時累積的成員

根的 rank 每升一級，都必須合併兩個同 rank 的不同集合：

- rank=0：至少 1 個成員。
- 兩個 rank=r−1 的根合併：各至少有 2^{r-1} 個成員，新根至少有 2^r 個。
- 不同 rank 合併只增加成員；FIND 不改變集合成員。

非根的 rank 固定在它失去根身分之時。**當時的集合至少有 2^r 個成員**，所以仍有 $2^r \le n$，即 $r \le \lfloor \log_2 n \rfloor$。這不是目前子樹大小的下界。

> 只按 rank 合併，單次 FIND 為 $O(\log n)$；再加路徑壓縮，改進的是**整段操作的攤銷界**。第一個深葉查找仍可能走 $\Theta(\log n)$ 條邊。

21.4 Rank 與路徑壓縮的精細分析

Ackermann 的迭代是函數合成

固定整段序列最終的元素數 n，令 $\alpha = \alpha(n)$。分析時不隨目前已建立的元素數重算 α。對 $j \geq 1$，定義

$$f^{(0)}(j) = j, \qquad f^{(i+1)}(j) = f(f^{(i)}(j)), \qquad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j) \ (k \geq 1).$$

上標表示**反覆把輸出當成下一次輸入**。由 $A_0^{(i)}(j) = j + i$ 得 $A_1(j) = 2j + 1$；對 i 歸納可得 $A_1^{(i)}(j) = 2^i(j + 1) - 1$，所以

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

各 A_k 對輸入嚴格遞增，且 $A_k(j) \geq j + 1$：由 A_0 開始歸納，較大的輸入會經過更多次、且每次都增加的迭代。又 $j + 1 \geq 2$，故 $A_k(j) > A_{k-1}(j)$。後面的分層界會用到這兩種單調性。

本章採用的 inverse-Ackermann 定義

採 Cormen 第三版 §21.4 的單參數版本：

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	大於 10^{80}

所以 $\alpha(2) = 0$、$\alpha(8) = 3$、$\alpha(2048) = 4$。不同教材可能採雙參數定義；不能直接混用其數值。

完整總界是 $O(m(1 + \alpha(n)))$，其中 m 次外部操作包含 n 次 MAKE-SET。只有 $n \geq 3$ 時才能簡寫為 $O(m\alpha(n))$；$n \leq 2$ 仍有常數操作成本。

哪些量會隨操作改變？

對操作數歸納可證：根以 parent 指向自己；非根的 rank 嚴格小於父 rank。相同 rank 的兩根合併時，保留根的 rank 加一；不同 rank 合併時，把小 rank 根接到大 rank 根。壓縮只改 parent，不改 rank。

節點一旦成為非根，其 rank 就固定。 後續它所指父節點的 rank 可能增加，或因壓縮改指更高的祖先；兩者都不會降低父 rank。這才是 level 不下降的依據。

rank 為 r 的根至少包含 2^r 個元素。非根沿用失去根身分時的歷史下界，故任一節點的 rank 至多 $\lfloor \log_2 n \rfloor < n$。這不是壓縮後的目前子樹大小。

接下來只對 rank 正、且不是根的節點定義 level 與 iter；rank 0 和根另外計費。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

level 決定層級，iter 記錄層內進度

令 $r = \mathrm{rank}(x) \geq 1$、$p = \mathrm{rank}(\mathrm{parent}(x))$，定義

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

正文以 $\ell = \mathrm{level}$、$i = \mathrm{iter}$ 縮短公式。先找父 rank 能跨過的最高層，再算該層可迭代幾次。

因 $p \geq r + 1 = A_0(r)$，有 $\ell \geq 0$。又 $A_\alpha(r) \geq A_\alpha(1) \geq n > p$，故 $\ell < \alpha$。定義給 $i \geq 1$；若 $i \geq r + 1$，就會有

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

與 ℓ 的最大性矛盾。因此 $0 \leq \ell < \alpha$、$1 \leq i \leq r$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用非負位能記錄尚可支付的工作

逐節點定義位能，再加總成整座森林的位能：

$$\phi(x) = \begin{cases} \alpha r, & x \text{ 是根或 } r = 0, \\ (\alpha - \ell(x))r - i(x), & x \text{ 非根且 } r \geq 1, \end{cases} \qquad \Phi = \sum_x \phi(x).$$

非根的下界為 $(\alpha - (\alpha - 1))r - r = 0$，上界為 $\alpha r - 1$。根的位能是 αr，rank 0 的位能為 0；所以 Φ 永遠非負。

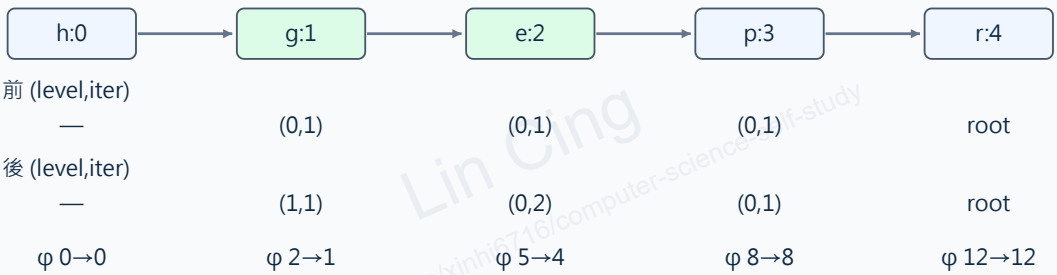
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五點路徑的位能帳

取 $n = 16$、$\alpha = 3$，壓縮 rank 依序為 0、1、2、3、4 的路徑。圖中的根保持自指。

FIND 的數值：level 與 iter 各自的前進

n=16, α=3；FIND(h) 後 h,g,e,p 都指根 r，rank 全部不變。



綠節點 g,e 是好節點：本path位能27→25；rank0、root、最後同level點為例外。

兩個好節點各釋放 1：g 靠 level 上升，e 靠同層 iter 上升。其餘三點位能不變，所以整條路徑位能由 27 降至 25。

level 上升時，iter 為何可以下降？

固定非根的 rank r。同一層內，父 rank 增加只會讓 iter 不變或增加；每多一次迭代，ϕ 就減一。

若層級由 k 升到 $k' > k$，iter 可能由 i 重設為較小的 i'。然而

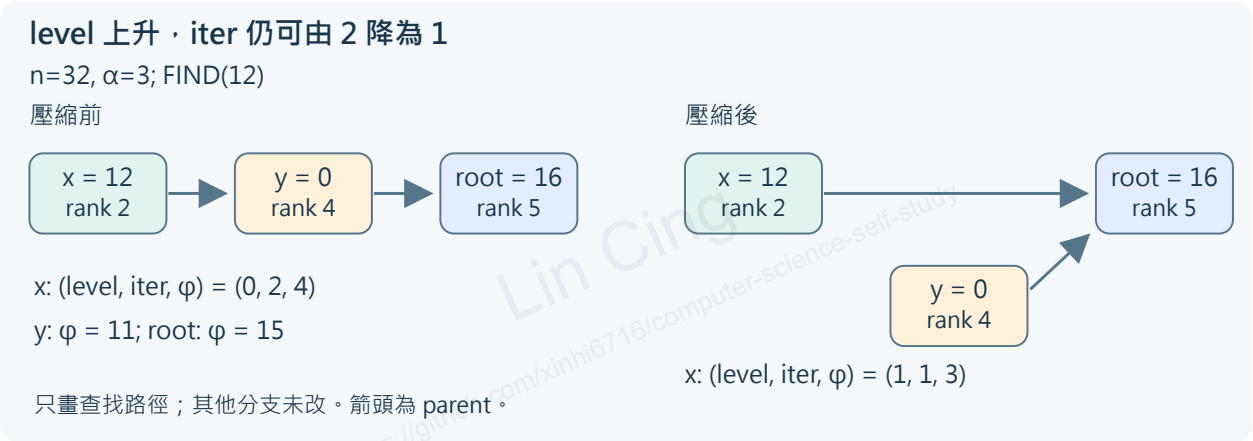
$$\Delta\phi = -(k' - k)r + (i - i') \le -r + (r - 1) = -1.$$

不等式用了 $k' - k \ge 1$、$i \le r$、$i' \ge 1$。**跨層的位能下降，足以支付 iter 重設。** 因此需要單調的是「先比較 level，再比較 iter」的字典序（lexicographic order），不是兩個座標各自遞增。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個 iter 確實下降的合法 FIND

取 $n = 32$、$\alpha = 3$。查找路徑是節點 $12 \to 0 \to 16$，rank 依序為 $2, 4, 5$。圖只保留這次路徑，其他分支不變。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨層數例的逐項計算

對 $x = 12$，固定 rank 為 2；壓縮只把父 rank 由 4 換成 5。

量	壓縮前	壓縮後
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$，故為 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$，故為 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$，故為 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$，故為 1
$\phi(x)$	$(3-0)2 - 2 = 4$	$(3-1)2 - 1 = 3$

節點 0 的位能保持 11，根的位能保持 15；其餘分支都不變。因此全森林 $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$。這裡只列路徑的加總；並未假設其他 29 個節點的位能為零。

如何真的建立這條路徑？

下列只用 MAKE-SET、UNION、FIND；同 rank 保留第一個根。先建立鍵 0 到 31 的單點。

操作	可核對的結果
對 0..15 分輪合併相鄰、同大小區塊	根 0 的 rank 為 4；節點 12 的 rank 為 2
FIND(12)	把 12 的 parent 改為 0；不改 rank
同法合併 16..31	根 16 的 rank 為 4
UNION(16,0)	根 16 升為 rank 5；0 的 parent 為 16
再次 FIND(12)	沿 $12 \to 0 \to 16$；12 的 parent 改為 16

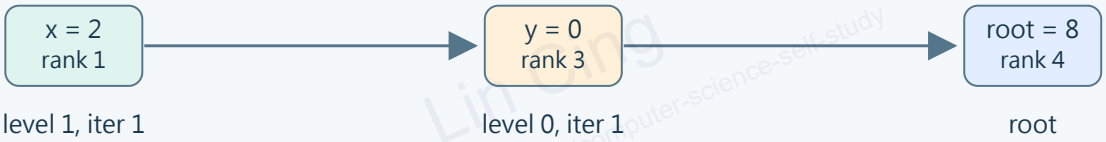
例如前半部的輪次是先合併 (0,1)、(2,3)、…、(14,15)，再合併 (0,2)、(4,6)、…，接著 (0,4)、(8,12)，最後 (0,8)。這說明 rank 數值來自合法歷史，沒有任意指定節點的 rank。

沿路的 level 不必遞增

取 $n = 16$。先把 0..7 合成根 0、rank 3 的集合並執行 FIND(2)；再把 8..15 合成根 8、rank 3 的集合，執行 UNION(8,0)。得到以下合法路徑。

rank 遞增，不代表 level 也遞增

n=16, α=3; FIND(2)



$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

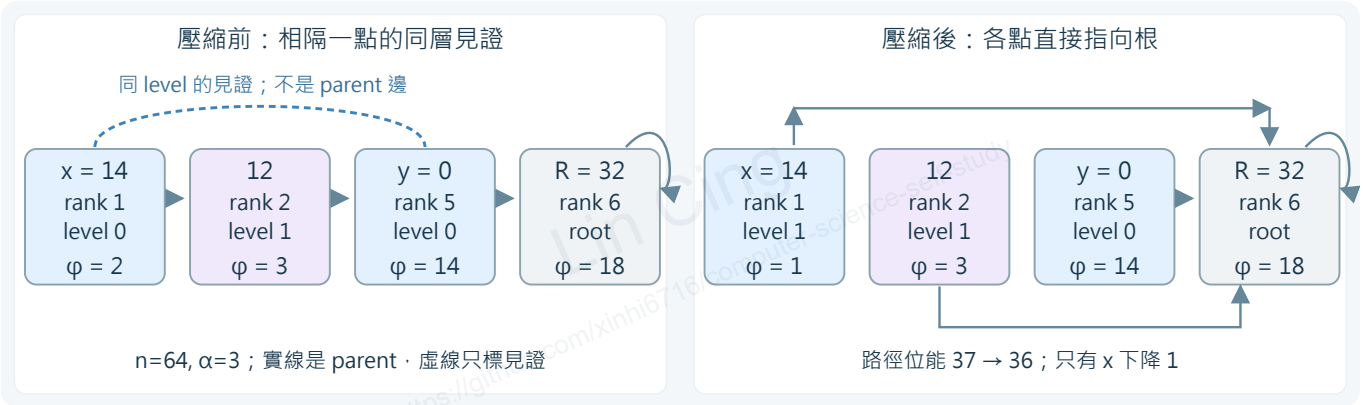
沿路 rank: 1 < 3 < 4；level: 1 > 0。

同 level 的最後一點才是例外；不用假設 level 沿路排序。

不同節點的 rank 也變了；不能把沿路比較，混同「同一節點隨時間」的單調性。

好節點的同層見證可以隔著其他 level

在壓縮前的路徑上，rank 大於 0 的非根節點 x 的後方若有另一個非根 y，且 $\ell(x) = \ell(y)$，就稱 x 為好節點。**找同層的後繼，不是只比較下一個節點。**



路徑 $14 \to 12 \to 0 \to 32$ 的 rank 為 1、2、5、6，level 為 0、1、0（根不定義 level）。14 的見證是 0；中間的 12 不必同層。兩個同層點不必在原樹中相鄰，壓縮後仍能替 14 帶來更高的父 rank。

先用合法合併建立這條路徑

取 64 個單點，同 rank 保留 UNION 的第一個根。「分輪建立區間」表示先合併相鄰單點，再合併相鄰同大小區塊。

操作	能從操作歷史核對的事實
分輪建立 0..31	根 0 的 rank=5；節點 12 的 rank=2，14 的 rank=1
FIND(12)	原路徑 $12 \to 8 \to 0$；只把 12 改接 0，14 仍接 12
分輪建立 32..63	根 32 的 rank=5
UNION(32,0)	兩根同 rank，32 升到 6；0 改接 32
尚未 FIND(14)	路徑為 $14 \to 12 \to 0 \to 32$，rank 為 1、2、5、6

FIND(12) 沿 $12 \to 8 \to 0$ 往上找到根，再把這條路上的父指標直接接到根。它不走進子樹，所以 14 仍接 12。之後的合併才把根 0 接到根 32；這些 rank 都來自合法操作，根 32 保持自指。

把每個節點的 level、iter、位能算出來

固定 $n = 64$、$\alpha = 3$。只對rank 大於 0 的非根節點計算 level、iter；根用 $\phi = \alpha r$。

節點	rank	父 rank：前→後	level：前→後	iter：前→後	ϕ：前→後
14	1	2→6	0→1	1→1	2→1
12	2	5→6	1→1	1→1	3→3
0	5	6→6	0→0	1→1	14→14
32（根）	6	6→6	不適用	不適用	18→18

對 14，壓縮前 $A_0(1) = 2 \le 2 < A_1(1) = 3$，故 level=0、iter=1。壓縮後 $A_1(1) = 3 \le 6 < A_2(1) = 7$，且 $A_1^{(2)}(1) = 7 > 6$，故 level=1、iter=1。位能由 $(3-0)1 - 1 = 2$ 降至 $(3-1)1 - 1 = 1$。

對 12，兩次皆有 $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$，故 level=1、iter=1。對 0，$A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$，故 level=0、iter=1、位能 14。路徑總和 $37 \to 36$；其餘點的 rank、父 rank 與根身分不變，故**全森林位能也恰減 1**。

好節點證明：每一步不等式各用了什麼？

所有量先固定在壓縮前。記 $\rho(v) = \text{rank}(v)$、$p_x = \text{parent}(x)$、$p_y = \text{parent}(y)$，根為 R。令 $r = \rho(x)$、$k = \ell(x) = \ell(y)$、$i = i(x)$。

逐步推導	理由
$\rho(R) \geq \rho(p_y)$	p_y 在通往根的路上，rank 沿非根 parent 邊嚴格增加
$\rho(p_y) \geq A_k(\rho(y))$	$\ell(y) = k$ 的定義
$A_k(\rho(y)) \geq A_k(\rho(p_x))$	y 就是 p_x 或其祖先，故 $\rho(y) \geq \rho(p_x)$；A_k 遞增
$A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$	iter 定義給 $\rho(p_x) \geq A_k^{(i)}(r)$，再用 A_k 遞增
$A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$	函數再合成一次，不是數值相乘

因此新父的 rank 至少為 $A_k^{(i+1)}(r)$。這裡沒有使用「路徑上的 level 遞增」；只使用 rank 的祖先次序與同一個 k 的函數單調性。

跨過下一個門檻，為何一定能支付工作？

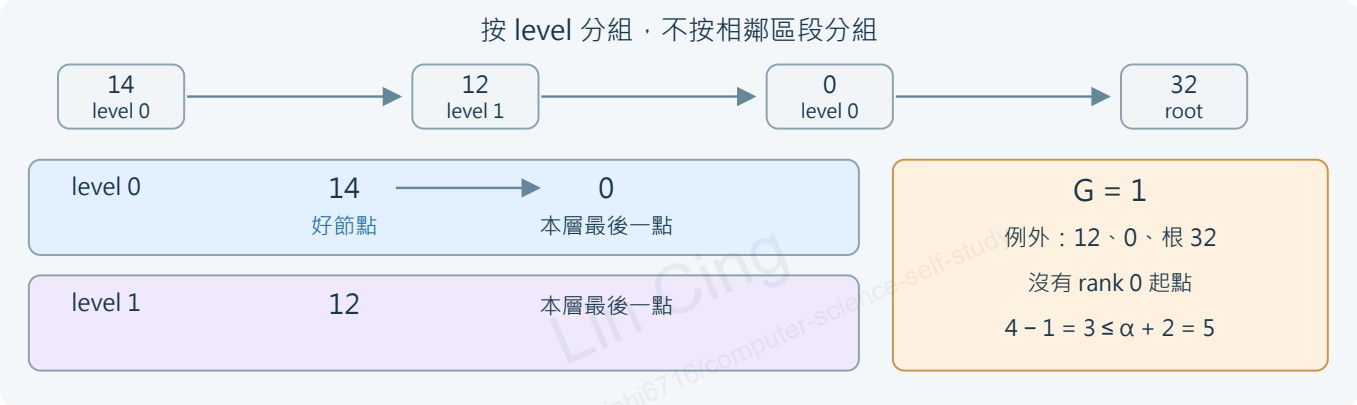
壓縮使 x 的父變成 R，但 r 不變。前頁證明的是新父 rank 跨過**舊 level 的下一次迭代門檻**，不保證新的 iter 必定比舊值大；必須分兩種情況。

壓縮後	位能的差	可以支付的原因
level 仍為 k	$\Delta\phi = -(i' - i) \leq -1$	iter 至少變成 $i + 1$
level 升為 $k' > k$	$\Delta\phi = -(k' - k)r + i - i'$	$k' - k \geq 1, i \leq r, i' \geq 1$，故 $\Delta\phi \leq -r + (r - 1) = -1$

在 64 點例子取 $x = 14, y = 0, k = 0, i = 1$，不等式數值為 $6 \geq 6 \geq 6 \geq 3 \geq 3$。舊層下一個門檻是 $A_0^{(2)}(1) = 3$；新父 rank=6 更跨過 $A_1(1) = 3$，所以落在第二種情況，位能下降 1。

這筆帳按「被壓縮的好節點 x」計費，不向見證 y 再收一次，也不要求 y 的位能下降。所有非好節點的位能亦不增加，故可以把各個好節點的下降量相加。

每個 level 只留下最後一點



沿原路徑向根看，同一 level 除了最後一點外，每點都有後方的同層見證。因此 14 可計費，12 與 0 分別是 level 1、0 的最後一點；再加根 32，共 3 個例外。若把三個相鄰區段 0／1／0 當成三組，就會漏算好節點 14。

從分組計數回到位能總下降

一般路徑含 s 個節點。rank 大於 0 的非根節點的 level 只能是 $0, \dots, \alpha - 1$，故各層最後一點至多 α 個；另加根與可能的 rank 0 起點，例外至多 $\alpha + 2$。rank 沿路嚴格增加，rank 0 起點最多一個。

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

G 是實際好節點數，g 只是保守下界。64 點例子 $s = 4, \alpha = 3$，所以 $g = 0$，實際卻有 $G = 1$、$\Delta\Phi = -1$。短路徑的下界可以不緊；不等式並未要求實際下降量必須等於下界。後面會用這個一般下界支付長路徑的成本。

用固定常數把路徑工作付清

設兩趟查找的實際成本至多 cs，固定 $C > c$ 並用放大位能 $\Psi = C\Phi$。則攤銷成本

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

路徑情況	代入 g 後的上界
$s \le \alpha + 2$	$cs \le c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$

因此每次 FIND 的攤銷成本為 $O(1 + \alpha)$。例如 $s = 7$、$\alpha = 3$、$c = 2$、$C = 3$，下界 $g = 2$，故 $\hat{c} \le 14 - 6 = 8$。C 在整段分析中固定，不能每次臨時換值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK 把位能增加限制在新根

MAKE-SET 新增 rank 0 節點，其位能為 0，攤銷成本仍為 $O(1)$。對 LINK，設舊根 u 接到保留根 v：

可能改變的點	位能變化的原因
u	從根變非根；rank 不變，位能不增
v 原有的孩子	父 rank 不降；前述單調性保證位能不增
v	不同 rank 時不變；同 rank 時加一，位能只增 α

其他點的 rank、父 rank 與根身分皆不變，所以位能不變。於是 $\Delta\Psi \le C\alpha$，LINK 的攤銷成本為 $O(1) + C\alpha = O(1 + \alpha)$。UNION 先做兩次 FIND，根不同才 LINK；最多拆成三次操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從每次攤銷界回到整段實際成本

令第 j 次實際成本為 c_j，攤銷成本為 $a_j = c_j + \Psi_j - \Psi_{j-1}$。相鄰位能相消後，

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \le \sum_{j=1}^{m'} a_j.$$

不等號用了初始空森林 $\Psi_0 = 0$ 與終態位能非負。拆開 UNION 後 $m' \le 3m$，所以原本 m 次操作的總成本為 $O(m(1+\alpha(n)))$。

這是整段序列的最壞總界。rank 與壓縮並用時，單次 FIND 仍可能走 $\Theta(\log n)$；先建立平衡的 rank 樹、再首次查找深葉即可達到。線性長鏈需要移除 rank 規則，不能當成本定理的單次反例。

位能分析參考：Cormen 等，第三版 §21.4，印刷頁 576–581。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study